

```

import pandas as pd
import seaborn as sns
from scipy.stats import zscore

df = sns.load_dataset("diamonds")

print("Descriptive Statistics")
print(df.describe())

print("\nSkewness")
skewness = df.select_dtypes(include='number').skew()
print(skewness)

print("\nSkewed Columns (|Skewness| > 1)")
skewed_cols = skewness[abs(skewness) > 1]
print(skewed_cols)

numeric_cols = df.select_dtypes(include='number').columns

df_zscore = df.copy()
df_zscore[numeric_cols] = df[numeric_cols].apply(zscore)

print("\nZ-Score Normalized Data")
print(df_zscore.head())

print("\nPearson Correlation")
pearson_corr = df[numeric_cols].corr(method='pearson')
print(pearson_corr)

print("\nSpearman Correlation")
spearman_corr = df[numeric_cols].corr(method='spearman')
print(spearman_corr)

```

```

Descriptive Statistics

```

	carat	depth	table	price
x \				
count	53940.000000	53940.000000	53940.000000	53940.000000
mean	0.797940	61.749405	57.457184	3932.799722
std	0.474011	1.432621	2.234491	3989.439738
min	0.200000	43.000000	43.000000	326.000000
25%	0.400000	61.000000	56.000000	950.000000
50%	0.700000	61.800000	57.000000	2401.000000
75%	1.040000	62.500000	59.000000	5324.250000
	6.540000			

```
max          5.010000      79.000000      95.000000  18823.000000
10.740000
```

```
          y          z
count  53940.000000  53940.000000
mean    5.734526    3.538734
std     1.142135    0.705699
min     0.000000    0.000000
25%     4.720000    2.910000
50%     5.710000    3.530000
75%     6.540000    4.040000
max     58.900000    31.800000
```

Skewness

```
carat    1.116646
depth   -0.082294
table    0.796896
price    1.618395
x        0.378676
y        2.434167
z        1.522423
dtype: float64
```

Skewed Columns (|Skewness| > 1)

```
carat    1.116646
price    1.618395
y        2.434167
z        1.522423
dtype: float64
```

Z-Score Normalized Data

```
          carat      cut color clarity      depth      table      price
x \
0 -1.198168    Ideal      E     SI2 -0.174092 -1.099672 -0.904095 -
1.587837
1 -1.240361  Premium      E     SI1 -1.360738  1.585529 -0.904095 -
1.641325
2 -1.198168     Good      E     VS1 -3.385019  3.375663 -0.903844 -
1.498691
3 -1.071587  Premium      I     VS2  0.454133  0.242928 -0.902090 -
1.364971
4 -1.029394     Good      J     SI2  1.082358  0.242928 -0.901839 -
1.240167
```

```
          y          z
0 -1.536196 -1.571129
1 -1.658774 -1.741175
2 -1.457395 -1.741175
3 -1.317305 -1.287720
4 -1.212238 -1.117674
```

Pearson Correlation

	carat	depth	table	price	x	y
z						
carat	1.000000	0.028224	0.181618	0.921591	0.975094	0.951722
	0.953387					
depth	0.028224	1.000000	-0.295779	-0.010647	-0.025289	-0.029341
	0.094924					
table	0.181618	-0.295779	1.000000	0.127134	0.195344	0.183760
	0.150929					
price	0.921591	-0.010647	0.127134	1.000000	0.884435	0.865421
	0.861249					
x	0.975094	-0.025289	0.195344	0.884435	1.000000	0.974701
	0.970772					
y	0.951722	-0.029341	0.183760	0.865421	0.974701	1.000000
	0.952006					
z	0.953387	0.094924	0.150929	0.861249	0.970772	0.952006
	1.000000					

Spearman Correlation

	carat	depth	table	price	x	y
z						
carat	1.000000	0.030104	0.194980	0.962883	0.996117	0.995572
	0.993183					
depth	0.030104	1.000000	-0.245061	0.010020	-0.023442	-0.025425
	0.103498					
table	0.194980	-0.245061	1.000000	0.171784	0.202231	0.195734
	0.159878					
price	0.962883	0.010020	0.171784	1.000000	0.963196	0.962719
	0.957232					
x	0.996117	-0.023442	0.202231	0.963196	1.000000	0.997895
	0.987355					
y	0.995572	-0.025425	0.195734	0.962719	0.997895	1.000000
	0.987068					
z	0.993183	0.103498	0.159878	0.957232	0.987355	0.987068
	1.000000					

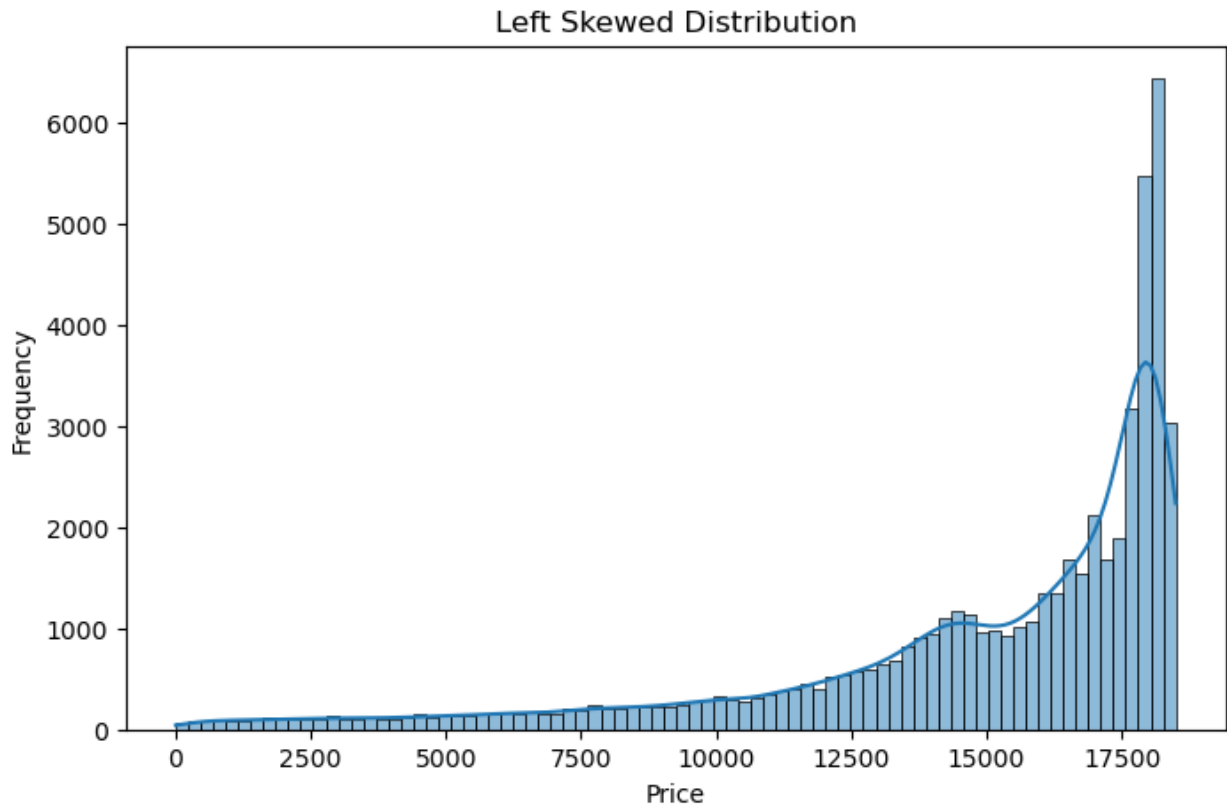
```
import matplotlib.pyplot as plt
import seaborn as sns

left_skew = df["price"].max() - df["price"]

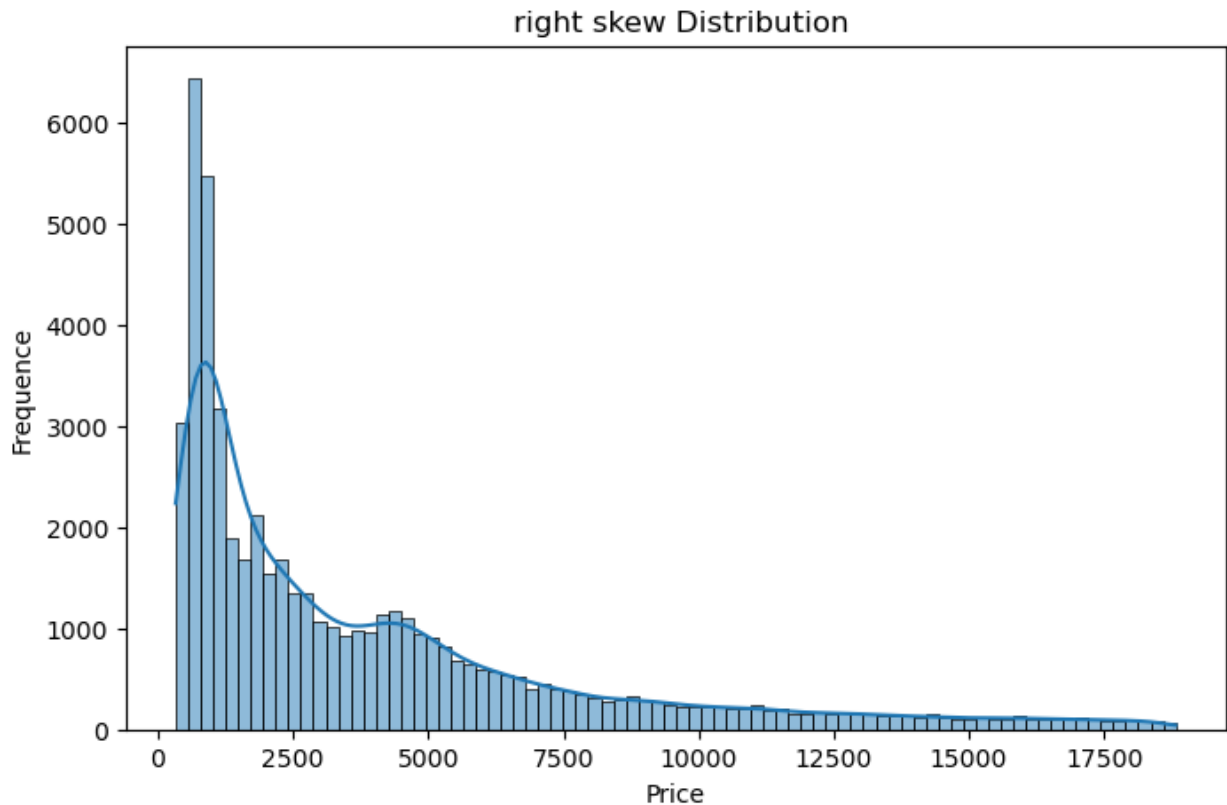
plt.figure(figsize=(8,5))
sns.histplot(left_skew, kde=True)

plt.title("Left Skewed Distribution")
plt.xlabel("Price")
plt.ylabel("Frequency")

plt.show()
```



```
#identify skewed distiribution(right)
df=sns.load_dataset("diamonds")
#histogram + KDE
plt.figure(figsize=(8,5))
sns.histplot(df["price"],kde=True)
plt.title(" right skew Distribution")
plt.xlabel("Price")
plt.ylabel("Frequence")
plt.show()
```



```
price_cols = df.select_dtypes(include=['number'])
zscore_df = price_cols.copy()
for col in zscore_df.columns:
    zscore_df[col] = (
        zscore_df[col] - zscore_df[col].mean()
    ) / zscore_df[col].std()
print(zscore_df.head())
```

	carat	depth	table	price	x	y
z						
0	-1.198157	-0.174090	-1.099662	-0.904087	-1.587823	-1.536181
1	-1.240350	-1.360726	1.585514	-0.904087	-1.641310	-1.658759
2	-1.198157	-3.384987	3.375631	-0.903836	-1.498677	-1.457382
3	-1.071577	0.454129	0.242926	-0.902081	-1.364959	-1.317293
4	-1.029384	1.082348	0.242926	-0.901831	-1.240155	-1.212227

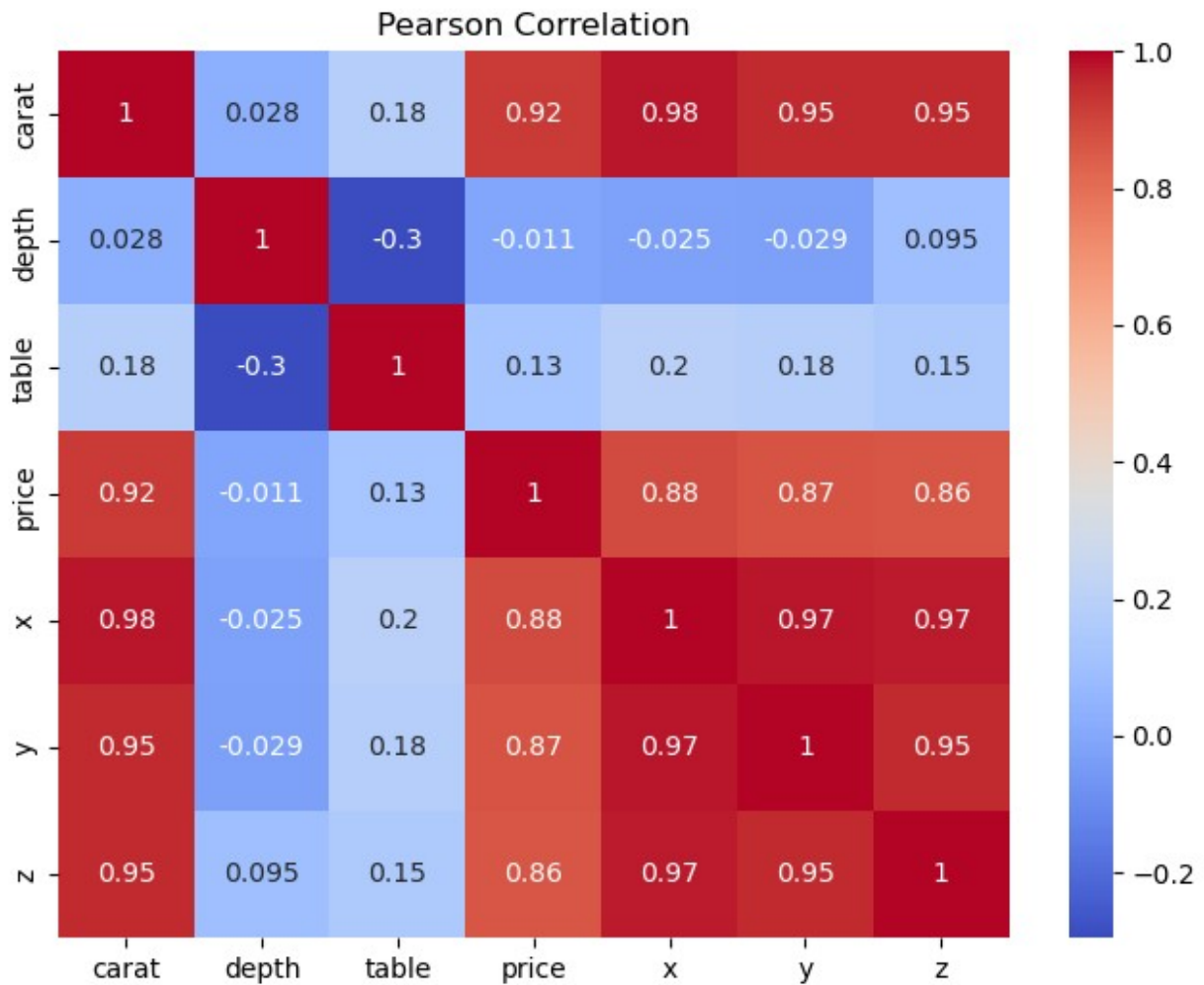
```
#heatmap
pearson_cor = df.corr(numeric_only=True)
plt.figure(figsize=(8,6))
sns.heatmap(
```

```

    pearson_cor,
    annot=True,
    cmap='coolwarm'
)

plt.title("Pearson Correlation")
plt.show()

```



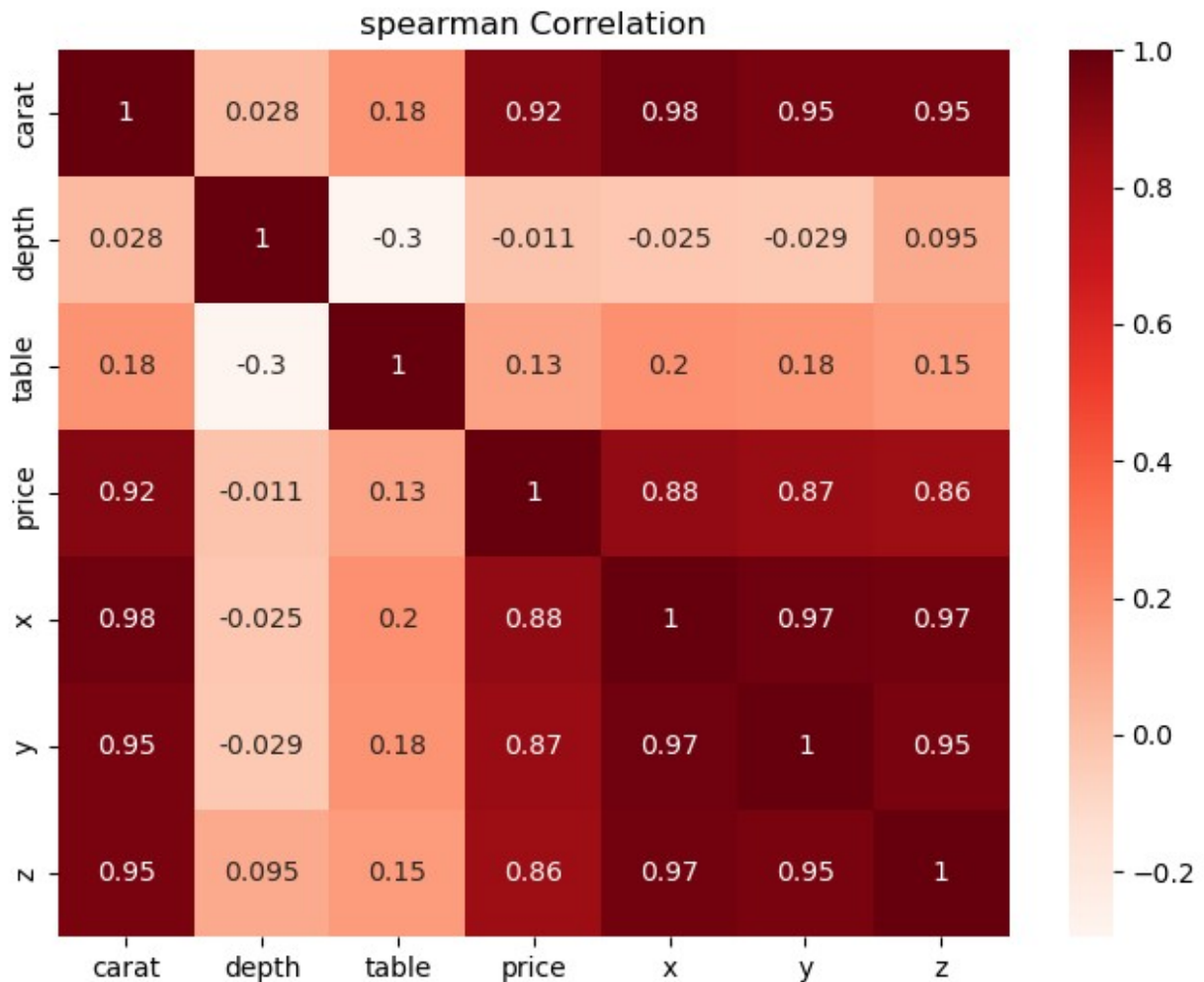
```

spearman_corr = price_cols.corr(method='spearman')
print(spearman_corr)
plt.figure(figsize=(8,6))
sns.heatmap(
    pearson_cor,
    annot=True,
    cmap='Reds'
)

```

```
plt.title("spearman Correlation")
plt.show()
```

	carat	depth	table	price	x	y
carat	1.000000	0.030104	0.194980	0.962883	0.996117	0.995572
depth	0.030104	1.000000	-0.245061	0.010020	-0.023442	-0.025425
table	0.194980	-0.245061	1.000000	0.171784	0.202231	0.195734
price	0.962883	0.010020	0.171784	1.000000	0.963196	0.962719
x	0.996117	-0.023442	0.202231	0.963196	1.000000	0.997895
y	0.995572	-0.025425	0.195734	0.962719	0.997895	1.000000
z	0.993183	0.103498	0.159878	0.957232	0.987355	0.987068



```
comparison = pd.DataFrame({
    "pearson": pearson_cor["price"],
    "spearman": spearman_corr["price"]
})
print(comparison)
```

	pearson	spearman
carat	0.921591	0.962883
depth	-0.010647	0.010020
table	0.127134	0.171784
price	1.000000	1.000000
x	0.884435	0.963196
y	0.865421	0.962719
z	0.861249	0.957232

```
import pandas as pd
import seaborn as sns
#load titanic data dataset
df = sns.load_dataset("titanic")
print(df.head())
print(df.shape)
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked
class \								
0	0	3	male	22.0	1	0	7.2500	S
Third								
1	1	1	female	38.0	1	0	71.2833	C
First								
2	1	3	female	26.0	0	0	7.9250	S
Third								
3	1	1	female	35.0	1	0	53.1000	S
First								
4	0	3	male	35.0	0	0	8.0500	S
Third								

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

(891, 15)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
```



```

0   survived      891 non-null    int64
1   pclass        891 non-null    int64
2   sex           891 non-null    object
3   age           714 non-null    float64
4   sibsp         891 non-null    int64
5   parch         891 non-null    int64
6   fare          891 non-null    float64
7   embarked      889 non-null    object
8   class         891 non-null    category
9   who           891 non-null    object
10  adult_male    891 non-null    bool
11  deck          203 non-null    category
12  embark_town   889 non-null    object
13  alive         891 non-null    object
14  alone         891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB

```

#1.inconsistent format

```

df.loc[0,'embark_town'] = 'southampton'
df.loc[1,'embark_town'] = 'SOUTHAMPTON'
df.loc[2,'embark_town'] = 'SOUTHampton'
print(df['embark_town'].unique())

```

```

['southampton' 'SOUTHAMPTON' 'SOUTHampton' 'Southampton' 'Queenstown'
 'Cherbourg' nan]

```

```

df['embark_town'] = df['embark_town'].str.title()

```

```

print(df['embark_town'].unique())

```

```

['Southampton' 'Queenstown' 'Cherbourg' nan]

```

```

print(df['embark_town'].isnull().sum())

```

```

2

```

```

df['embark_town'] = df['embark_town'].fillna('Kingstown')

```

```

print(df['embark_town'].unique())

```

```

['Southampton' 'Queenstown' 'Cherbourg' 'Kingstown']

```

duplicate rows

```

duplicate_rows = df.iloc[:5]
df_dup = pd.concat([df,duplicate_rows])
print(df_dup.shape)

```

```

(896, 15)

```

#To find duplicates

```

print(df_dup.duplicated().sum())

```

112

#view duplicate rows

```
df_dup[df_dup.duplicated()]
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked
class \								
47 Third	1	3	female	NaN	0	0	7.7500	Q
76 Third	0	3	male	NaN	0	0	7.8958	S
77 Third	0	3	male	NaN	0	0	8.0500	S
87 Third	0	3	male	NaN	0	0	8.0500	S
95 Third	0	3	male	NaN	0	0	8.0500	S
..	...	...	...	...	...	...	...	...
...								
0 Third	0	3	male	22.0	1	0	7.2500	S
1 First	1	1	female	38.0	1	0	71.2833	C
2 Third	1	3	female	26.0	0	0	7.9250	S
3 First	1	1	female	35.0	1	0	53.1000	S
4 Third	0	3	male	35.0	0	0	8.0500	S

	who	adult_male	deck	embark_town	alive	alone
47	woman	False	NaN	Queenstown	yes	True
76	man	True	NaN	Southampton	no	True
77	man	True	NaN	Southampton	no	True
87	man	True	NaN	Southampton	no	True
95	man	True	NaN	Southampton	no	True
..	...	...	...	...	...	...
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Southampton	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

[112 rows x 15 columns]

```
df_dup=df_dup.drop_duplicates()
```

```
print(df_dup.duplicated().sum())
```

0

```

#3. typos
df.loc[0, 'embark_town'] = 'Southampton'
df.loc[0, 'embark_town'] = 'Southhampton'

print(df['embark_town'].value_counts())

embark_town
Southampton      644
Cherbourg         167
Queenstown        77
Kingstown          2
Southhampton       1
Name: count, dtype: int64

#fix typos
df['embark_town'] = df['embark_town'].replace({
    'Southampton' : 'Southampton',
    'Southhampton' : 'Southampton'
})

#verify:

print(df['embark_town'].value_counts())

embark_town
Southampton      645
Cherbourg         167
Queenstown        77
Kingstown          2
Name: count, dtype: int64

print(sorted(df['embark_town'].dropna().unique()))

['Cherbourg', 'Kingstown', 'Queenstown', 'Southampton']

#unique mismatch
df.loc[0, 'age'] = 240
df.loc[1, 'age'] = 300

#detect unusual values
df['age'].describe()

count      714.000000
mean        30.371387
std         19.362012
min          0.420000
25%         20.125000
50%         28.000000
75%         38.750000
max         300.000000
Name: age, dtype: float64

```

```
#fix unit mismatch
##convert months to years.
df.loc[df["age"]>120,"age"]=(
    df.loc[df["age"]>120,"age"]/12
)
#verify
df["age"].describe()
```

```
count    714.000000
mean      29.678109
std       14.525902
min        0.420000
25%       20.000000
50%       28.000000
75%       38.000000
max       80.000000
Name: age, dtype: float64
```

```
#Wrong category labels
df["survived_label"] = df['survived'].map({
    0: "No",
    1: "Yes"
})
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked
class \								
0	0	3	male	20.0	1	0	7.2500	S
Third								
1	1	1	female	25.0	1	0	71.2833	C
First								
2	1	3	female	26.0	0	0	7.9250	S
Third								
3	1	1	female	35.0	1	0	53.1000	S
First								
4	0	3	male	35.0	0	0	8.0500	S
Third								

	who	adult_male	deck	embark_town	alive	alone	survived_label
0	man	True	NaN	Southampton	no	False	No
1	woman	False	C	Southampton	yes	False	Yes
2	woman	False	NaN	Southampton	yes	True	Yes
3	woman	False	C	Southampton	yes	False	Yes
4	man	True	NaN	Southampton	no	True	No

```
#check
df.isnull().mean()
```

```
survived    0.000000
pclass      0.000000
```

```

sex          0.000000
age          0.198653
sibsp       0.000000
parch       0.000000
fare        0.000000
embarked     0.002245
class       0.000000
who         0.000000
adult_male  0.000000
deck        0.772166
embark_town  0.000000
alive       0.000000
alone       0.000000
survived_label 0.000000
dtype: float64

```

```

#remove
df.drop(columns=["deck"],inplace=True)
df.head()

```

	survived	pclass	sex	age	sibsp	parch	fare	embarked
class \								
0 Third	0	3	male	20.0	1	0	7.2500	S
1 First	1	1	female	25.0	1	0	71.2833	C
2 Third	1	3	female	26.0	0	0	7.9250	S
3 First	1	1	female	35.0	1	0	53.1000	S
4 Third	0	3	male	35.0	0	0	8.0500	S

	who	adult_male	embark_town	alive	alone	survived_label
0	man	True	Southampton	no	False	No
1	woman	False	Southampton	yes	False	Yes
2	woman	False	Southampton	yes	True	Yes
3	woman	False	Southampton	yes	False	Yes
4	man	True	Southampton	no	True	No

```

#Methon 1: Mean Imputation
#replace muissing values with the average value
df["age"] = df["age"].fillna(df["age"].mean())
df.head(20)
#Methon 1: Median Imputation
#replace muissing values with the average value
df["age"] = df["age"].fillna(df["age"].median())
df.head(20)

```

	survived embarked \	pclass	sex	age	sibsp	parch	fare
0	0	3	male	20.000000	1	0	7.2500
1	1	1	female	25.000000	1	0	71.2833
2	1	3	female	26.000000	0	0	7.9250
3	1	1	female	35.000000	1	0	53.1000
4	0	3	male	35.000000	0	0	8.0500
5	0	3	male	29.678109	0	0	8.4583
6	0	1	male	54.000000	0	0	51.8625
7	0	3	male	2.000000	3	1	21.0750
8	1	3	female	27.000000	0	2	11.1333
9	1	2	female	14.000000	1	0	30.0708
10	1	3	female	4.000000	1	1	16.7000
11	1	1	female	58.000000	0	0	26.5500
12	0	3	male	20.000000	0	0	8.0500
13	0	3	male	39.000000	1	5	31.2750
14	0	3	female	14.000000	0	0	7.8542
15	1	2	female	55.000000	0	0	16.0000
16	0	3	male	2.000000	4	1	29.1250
17	1	2	male	29.678109	0	0	13.0000
18	0	3	female	31.000000	1	0	18.0000
19	1	3	female	29.678109	0	0	7.2250
	class	who	adult_male	embark_town	alive	alone	survived_label
0	Third	man	True	Southampton	no	False	No
1	First	woman	False	Southampton	yes	False	Yes
2	Third	woman	False	Southampton	yes	True	Yes

3	First	woman	False	Southampton	yes	False	Yes
4	Third	man	True	Southampton	no	True	No
5	Third	man	True	Queenstown	no	True	No
6	First	man	True	Southampton	no	True	No
7	Third	child	False	Southampton	no	False	No
8	Third	woman	False	Southampton	yes	False	Yes
9	Second	child	False	Cherbourg	yes	False	Yes
10	Third	child	False	Southampton	yes	False	Yes
11	First	woman	False	Southampton	yes	True	Yes
12	Third	man	True	Southampton	no	True	No
13	Third	man	True	Southampton	no	False	No
14	Third	child	False	Southampton	no	True	No
15	Second	woman	False	Southampton	yes	True	Yes
16	Third	child	False	Queenstown	no	False	No
17	Second	man	True	Southampton	yes	True	Yes
18	Third	woman	False	Southampton	no	False	No
19	Third	woman	False	Cherbourg	yes	True	Yes

```

#mode
df["embarked_mode"] = df["embarked"].fillna(df["embarked"].mode()[0])
print(df["embarked"].isnull().sum())
2

#Apply forward fill
df["embarked_ffill"] = df["embarked"].ffill()
#check missing value
print(df["embarked_ffill"].isnull().sum())
0

# 5 KNN Imputation
import pandas as pd

```

```
import numpy as np
data = {
    'age' : [22,34,np.nan,24,23],
    'fare' : [7.25,8.05,7.90,8.00,7.50]
}
df = pd.DataFrame(data)
print(df)
```

	age	fare
0	22.0	7.25
1	34.0	8.05
2	NaN	7.90
3	24.0	8.00
4	23.0	7.50

```
#Apply KNN Imputation
from sklearn.impute import KNNImputer
KNN = KNNImputer(n_neighbors=3)
df_KNN = pd.DataFrame(
    KNN.fit_transform(df),
    columns=df.columns)
print(df_KNN)
```

	age	fare
0	22.0	7.25
1	34.0	8.05
2	27.0	7.90
3	24.0	8.00
4	23.0	7.50

```
#check missing values
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer
#load Titanic dataset
df= sns.load_dataset("titanic")
#forward fill for embarked
df["embarked_ffill"] = df["embarked"].ffill()
#KNN Imutation for age
cols = [
    'age','fare','sibsp','parch','pclass']
imputer = KNNImputer(n_neighbors=5)
df[cols] = imputer.fit_transform(df[cols])
print(df.isnull().sum())
```

survived	0
pclass	0
sex	0
age	0


```
sibsp      0
parch      0
fare      0
embarked   2
class      0
who        0
adult_male 0
deck      688
embark_town 2
alive      0
alone      0
embarked_ffill 0
dtype: int64
```

#checking missingvalues

```
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer
#Load Tianic dataset
df=sns.load_dataset('titanic')
#forward Fill for embarked
df['embarked_ffill']=df['embarked'].ffill()
#KNN Imputation for age
cols=['age','fare','sibsp','parch','pclass']
imputer=KNNImputer(n_neighbors=5)
df[cols]=imputer.fit_transform(df[cols])
print(df.isnull().sum())
```

```
survived    0
pclass      0
sex         0
age         0
sibsp       0
parch       0
fare        0
embarked    2
class       0
who         0
adult_male  0
deck       688
embark_town 2
alive       0
alone       0
embarked_ffill 0
dtype: int64
```

```
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
df = sns.load_dataset('titanic')
```

```

le = LabelEncoder()
df['sex_encoded'] = le.fit_transform(
    df['sex']
)
df = pd.get_dummies(
    df,
    columns=['embarked'],
    drop_first=True
)
target_map = df.groupby('pclass')['survived'].mean()
df['class_target'] = (df['pclass'].map(target_map))
print(df.head())

```

	survived	pclass	sex	age	sibsp	parch	fare	class	who
0	0	3	male	22.0	1	0	7.2500	Third	man
1	1	1	female	38.0	1	0	71.2833	First	woman
2	1	3	female	26.0	0	0	7.9250	Third	woman
3	1	1	female	35.0	1	0	53.1000	First	woman
4	0	3	male	35.0	0	0	8.0500	Third	man

	adult_male	deck	embark_town	alive	alone	sex_encoded	embarked_Q
0	True	NaN	Southampton	no	False	1	False
1	False	C	Cherbourg	yes	False	0	False
2	False	NaN	Southampton	yes	True	0	False
3	False	C	Southampton	yes	False	0	False
4	True	NaN	Southampton	no	True	1	False

	embarked_S	class_target
0	True	0.242363
1	False	0.629630
2	True	0.242363
3	True	0.629630
4	True	0.242363

```

from sklearn.preprocessing import MinMaxScaler
scalar=MinMaxScaler()
df[['age_minmax','fare_minmax']]=scalar.fit_transform(
    df[['age','fare']])
df[['age_minmax','fare_minmax']].head()

```

	age_minmax	fare_minmax
0	0.271174	0.014151
1	0.472229	0.139136
2	0.321438	0.015469
3	0.434531	0.103644
4	0.434531	0.015713

```

from sklearn.preprocessing import minmax_scale
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("titanic")

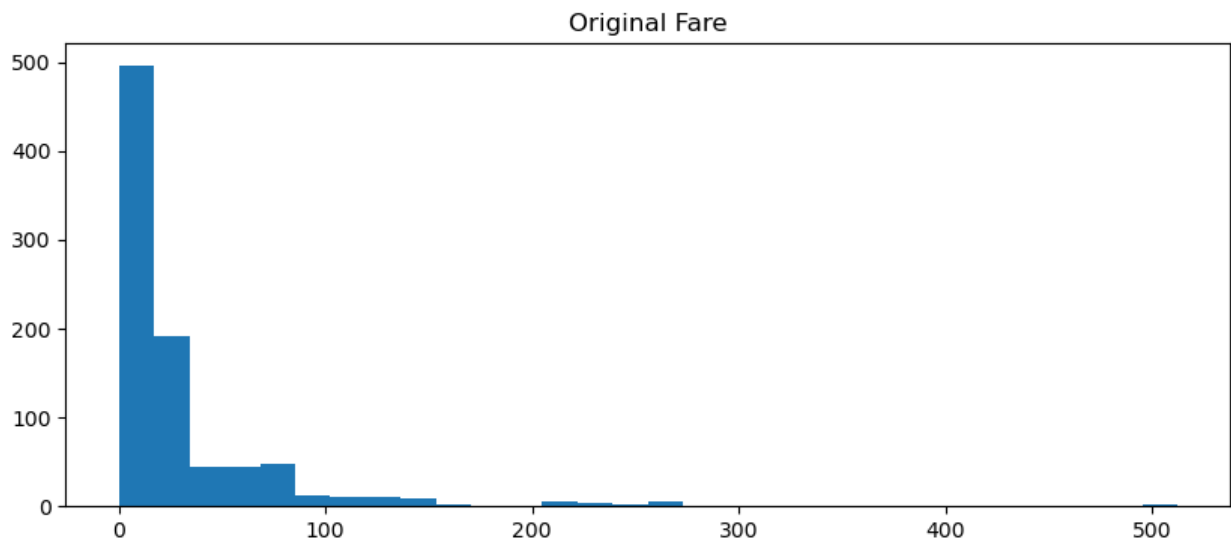
df["fare_minmax"] =
minmax_scale(df["fare"].fillna(df["fare"].median()))

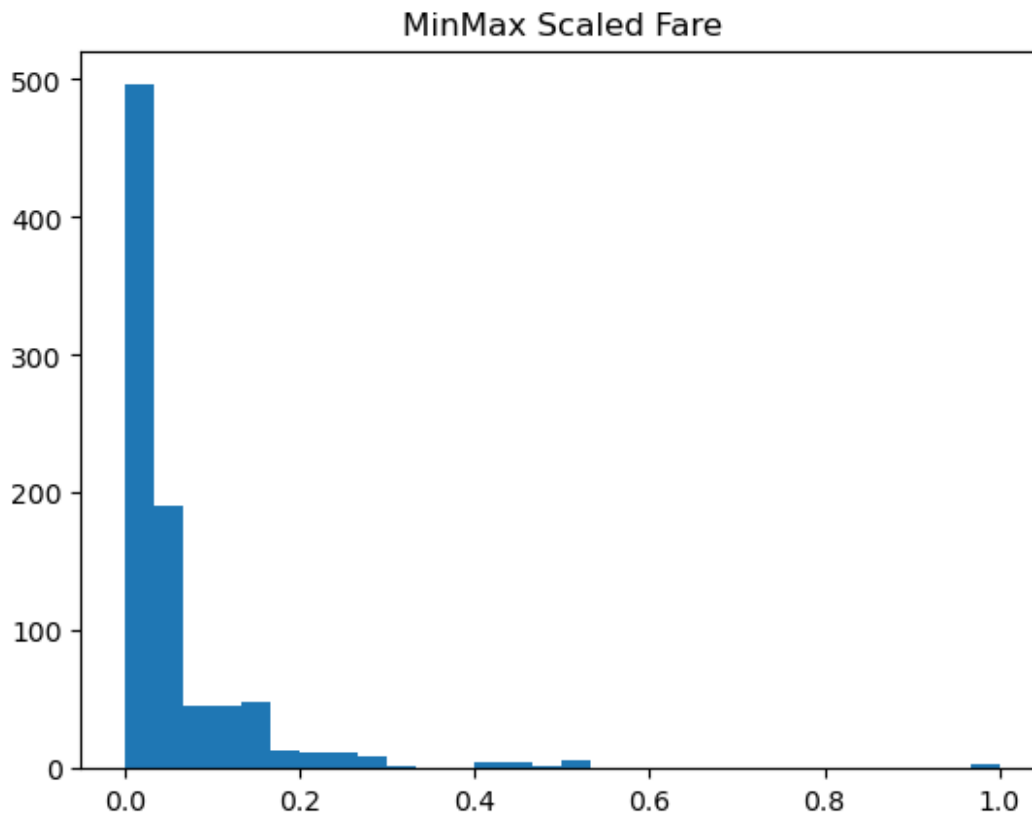
plt.figure(figsize=(10, 4))

plt.hist(df["fare"].dropna(), bins=30)
plt.title("Original Fare")
plt.show()

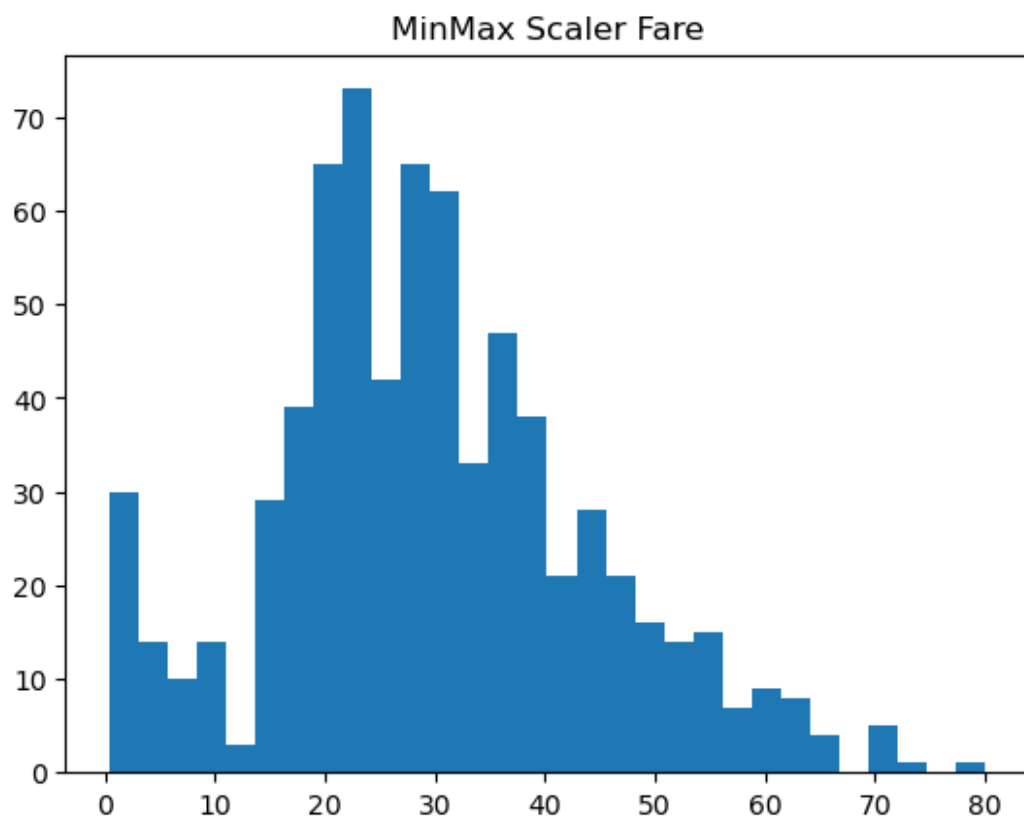
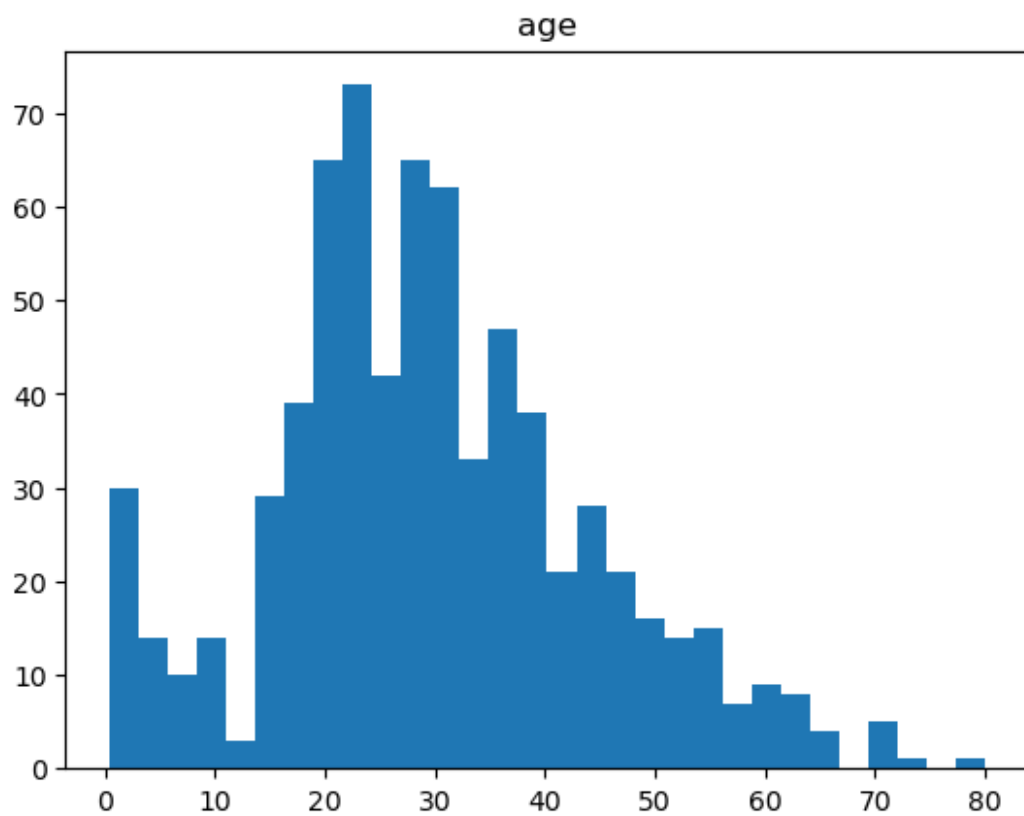
plt.hist(df["fare_minmax"], bins=30)
plt.title("MinMax Scaled Fare")
plt.show()

```





```
import matplotlib.pyplot as plt
plt.hist(df['age'].dropna(),bins=30)
plt.title("age")
plt.show()
plt.hist(df['age'],bins=30)
plt.title("MinMax Scaler Fare")
plt.show()
```



```
# robust Scaler
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['age_std', 'fare_std']] = scaler.fit_transform(
    df[['age', 'fare']]
)
df[['age_std', 'fare_std']].head()
```

	age_std	fare_std
0	-0.530377	-0.502445
1	0.571831	0.786845
2	-0.254825	-0.488854
3	0.365167	0.420730
4	0.365167	-0.486337